

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	S Susmitha	Reg. No.:	192571068
Section / Batch:	Slot B	Date:	27.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I S Susmitha (192571068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____



RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Name	Register Number	Course Code	Course
S Susmitha	192571068	CSA0603	Design and Analysis of Algorithms

Question:

Determine whether a graph $G = (V, E)$ can be properly colored using at most k colors, such that no two adjacent vertices share the same color. Clearly define constraints related to adjacency and color limits, analyze how constraints influence feasibility, develop a backtracking-based solution, apply constraint checking to ensure valid coloring, and justify correctness and NP-Complete classification.

K-COLORABILITY PROBLEM (NP-COMPLETE PROBLEM)

1. Problem Understanding

1.1 Aim and Objectives

To determine, for a range of structurally distinct graphs, whether a proper k -coloring exists, using a backtracking algorithm built on explicit adjacency and color-limit constraints, and to use the resulting behaviour to explain why the problem is classified as NP-Complete.

Specific objectives:

- Formally define the K-Colorability decision problem and the constraints that any valid solution must satisfy.
- Select a small set of structurally different test graphs — general, bipartite, complete, sparse, and infeasible — so the algorithm is exercised under contrasting conditions rather than a single convenient case.
- Implement a constraint-checked backtracking algorithm and trace its execution by hand and by program on at least one graph.
- Empirically measure how tightening the color budget k changes the amount of search performed, connecting the constraint-analysis discussion to real runtime data rather than only asymptotic claims.
- Establish the correctness of the algorithm and justify the problem's membership in the NP-Complete class.

1.2 Formal Definition

Given an undirected graph $G = (V, E)$ and a positive integer k , the K-Colorability decision problem asks whether there exists a function $\text{color}: V \rightarrow \{1, 2, \dots, k\}$ such that for every edge $(u, v) \in E$, $\text{color}(u) \neq \text{color}(v)$. If such a function exists, G is said to be k -colorable.

This is the decision version of the problem — it answers only yes/no for a given k . The related optimization version, finding the smallest k for which G is colorable (the chromatic number $\chi(G)$), is strictly harder in the sense that solving it requires solving the decision version repeatedly; this report focuses on the decision version, since that is the form in which NP-Completeness is normally established.

1.3 Background: P, NP, NP-Hard, and NP-Complete

The complexity classes relevant to this problem are defined as follows:

- P: the class of decision problems solvable by a deterministic algorithm in polynomial time.
- NP: the class of decision problems for which a proposed solution (a certificate) can be verified in polynomial time, even if finding that solution may take much longer.
- NP-Hard: a problem to which every problem in NP can be reduced in polynomial time; an NP-Hard problem is at least as hard as the hardest problems in NP, and need not itself be in NP.
- NP-Complete: a problem that is both in NP and NP-Hard — the intersection that represents the hardest problems whose solutions can still be verified quickly.

K-Colorability for $k \geq 3$ sits inside this last category, as formally justified in Section 5. This classification is exactly what motivates the use of backtracking in Section 3: since no known polynomial-time algorithm decides K-Colorability in general, an exhaustive-but-pruned search is the standard practical approach for small and medium instances.

1.3.1 K-Colorability as a Constraint Satisfaction Problem

The CO5 outcome for this assessment is naturalization — translating a real combinatorial statement into a formal model that can be reasoned about and solved. K-Colorability naturalizes cleanly into the standard Constraint Satisfaction Problem (CSP) framework used throughout artificial intelligence and operations research:

- Variables: one variable per vertex, $V = \{v_1, v_2, \dots, v_n\}$.
- Domains: every variable shares the same domain, the color set $\{1, 2, \dots, k\}$.
- Constraints: for every edge $(v_i, v_j) \in E$, a binary inequality constraint $v_i \neq v_j$.

Viewed this way, the backtracking algorithm developed in Section 3 is not specific to graph coloring at all — it is the generic CSP backtracking search (assign a variable, check constraints incrementally, backtrack on failure) specialized to inequality constraints on a graph's edges. This is the same underlying search that solves Sudoku (Section 4.2), the N-Queens problem, and many scheduling problems, which is why the same `isSafe()`-style constraint check reappears across such a wide range of applications. Recognizing K-Colorability as an instance of the general CSP model is what allows the correctness argument in Section 5.1 — which never mentions colors or edges specifically — to be reused, essentially unchanged, for any other CSP built on the same assign-check-backtrack pattern.

Historically, the best-known instance of this problem is map coloring: the four-color theorem (1976) established that $k = 4$ always suffices for any planar map, but proving that bound required one of the first major computer-assisted proofs in mathematics, precisely because no simple hand argument was found for it. This report's Graph 3 (K5) illustrates why: K5 is not planar and needs 5 colors, showing that the '4 colors suffice' guarantee is specific to planar structure and does not generalize to arbitrary graphs.

1.4 Selection and Visualization of Test Graphs

Rather than testing on a single convenient graph, five structurally different small graphs were chosen so that the algorithm's behaviour — and the constraints governing feasibility — can be compared directly across contrasting cases.

Graph	Structure	Reason for Inclusion
Graph 1	Triangle-forcing, $n = 5, k = 3$	The general case: a clique of size 3 forces at least 3 colors, exercising the core adjacency-check logic.
Graph 2	Even cycle (bipartite), $n = 6, k = 2$	Tests the easiest non-trivial case — an even cycle is always 2-colorable — confirming the algorithm does not overuse colors when few suffice.
Graph 3	Complete graph K5, $k = 5$	The extreme case where every vertex is mutually adjacent: k must equal n exactly, isolating the worst-case clique behaviour.
Graph 4	Sparse tree, $n = 7, k = 2$	Trees are always 2-colorable and have minimal branching; this is the best case for backtracking, with almost no pruning needed.
Graph 5	Same as Graph 1, $k = 2$ (deliberately insufficient)	Tests the boundary/failure case: the algorithm must exhaust the entire search space and correctly report that no valid coloring exists.

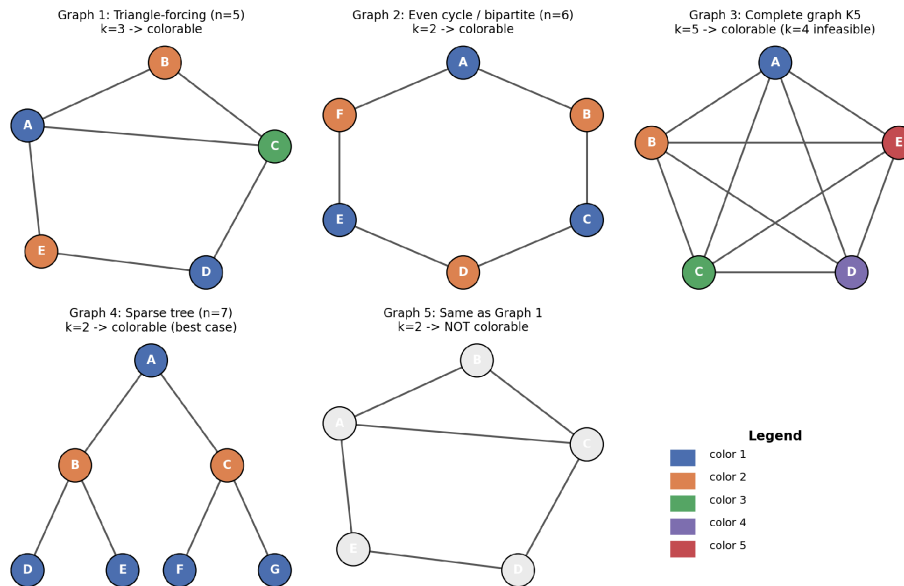


Figure 1: The five test graphs used throughout this report, each shown with a valid coloring where one exists. Graph 5 is shown uncolored since no valid 2-coloring exists for it.

2. Constraint Analysis

2.1 Constraints Governing a Valid Solution

Two hard constraints and one structural constraint define every accepted solution:

Constraint	Description	Type
Adjacency Constraint	For every edge (u, v) in E , the colors assigned to u and v must differ: $\text{color}(u) \neq \text{color}(v)$	Hard constraint
Color-Limit Constraint	Every vertex must be assigned a color from a fixed palette of size k : $\text{color}(v) \in \{1, 2, \dots, k\}$	Hard constraint
Completeness Constraint	Every vertex in V must receive exactly one color before the assignment is accepted as a solution	Structural constraint

2.2 How Graph Structure Influences Feasibility

The feasibility of a k -coloring depends directly on how the adjacency and color-limit constraints interact with the graph's structure:

- Cliques:** A clique of size m (a set of m mutually adjacent vertices) forces at least m distinct colors, since every pair within the clique is adjacent. This gives an immediate lower bound: k must be $\geq$ the size of the largest clique. Graphs 1, 3, and 5 are chosen specifically to test this bound at three different margins (met exactly, met with room to spare, and violated).
- Cycles:** An odd-length cycle cannot be 2-colored, since traversing it forces alternating colors that clash at the closing edge; an even-length cycle (Graph 2) is always 2-colorable.
- Degree of vertices:** A vertex with degree d can be forced into a corner if all d neighbors already occupy d distinct colors and $k = d$; the assignment then fails at that vertex and triggers backtracking. Low-degree graphs such as the tree in Graph 4 rarely trigger this.

- **Vertex ordering:** Coloring high-degree (heavily constrained) vertices first prunes the search tree earlier, since their neighbors' choices are restricted sooner rather than later — this is used implicitly whenever the fixed vertex order happens to place a clique's vertices consecutively, as in Graphs 1, 3, and 5.

These structural properties are what backtracking exploits: instead of blindly trying all k^n combinations, the algorithm checks the adjacency constraint incrementally and abandons a partial assignment the moment it becomes infeasible, well before all n vertices are colored.

2.3 Constraint Interaction Across the Five Test Graphs

The clique lower bound described above is directly testable: whenever the largest clique size exceeds k , no coloring can exist, regardless of how the rest of the graph is structured. The table below confirms this for every test graph.

Graph	Max clique	k given	Feasible?	Dominant constraint effect
Graph 1	3	3	Yes	Clique lower bound met exactly
Graph 2	2	2	Yes	No odd cycle, so 2 colors suffice
Graph 3	5	5	Yes ($k=4$ fails)	Clique size equals k exactly; $k-1$ is provably infeasible
Graph 4	2	2	Yes	Tree structure: minimal degree, near-zero pruning
Graph 5	3	2	No	Clique lower bound (3) exceeds available colors (2)

Graph 3 and Graph 5 isolate this boundary most clearly. In Graph 3, k is set exactly equal to the clique size (5), so the color-limit constraint is satisfied with no slack — reducing k by even 1 makes the graph provably uncolorable, since five mutually adjacent vertices cannot share fewer than five colors. In Graph 5, the same triangle structure as Graph 1 is kept, but k is deliberately reduced to 2, which is below the clique lower bound of 3 — this is used in Section 3.7 to show what the backtracking search does when no solution exists.

2.4 Constraint Tightness and Search Effort

Constraint analysis is not only about whether a solution exists, but about how much search is needed to find or rule one out. Section 3.8 measures this directly: when k is set just below a graph's true chromatic number ('tight'), the constraint check fails late and repeatedly, forcing extensive backtracking before the algorithm can conclude infeasibility. When k is set comfortably above the chromatic number ('loose'), the first few color choices tend to succeed and the recursion rarely needs to backtrack at all. This link between constraint tightness and search cost is one of the central empirical findings of this report.

3. Solution Development

3.1 Backtracking Algorithm — Concept and Steps

The backtracking algorithm attempts to color one vertex at a time, in a fixed order. At each vertex it tries every color from 1 to k , accepts the first color that does not conflict with already-colored neighbors, and recurses to the next vertex. If no color works at a vertex, or a deeper recursive call fails for every choice, the algorithm undoes (backtracks) the last assignment and tries the next available color at the previous vertex.

Pseudocode

```
function graphColoring(graph, k, colorArray, vertex):
    if vertex == n:           // all vertices colored
        return true          // solution found

    for c in 1 to k:
        if isSafe(graph, colorArray, vertex, c):
            colorArray[vertex] = c    // tentative assignment

            if graphColoring(graph, k, colorArray, vertex + 1):
                return true           // propagate success

            colorArray[vertex] = 0    // backtrack: undo assignment

    return false                // no color works at this vertex

function isSafe(graph, colorArray, vertex, c):
    for each neighbor u of vertex:
        if colorArray[u] == c:
            return false           // adjacency constraint violated
    return true
```

3.2 Constraint Checking (isSafe)

The `isSafe()` routine is the point at which the adjacency constraint is enforced. Before a color `c` is tentatively assigned to a vertex, the algorithm scans every neighbor of that vertex in the adjacency list. If any neighbor already holds color `c`, the assignment is rejected immediately and the next color is tried — the recursion is never allowed to proceed on an invalid partial coloring. This constraint check is what converts a brute-force enumeration of k^n colorings into a pruned search that discards an entire subtree the moment a conflict is detected.

3.3 Python Implementation

The script below runs the same backtracking function against all five test graphs and reports the resulting assignment (or infeasibility) along with the number of color-checks performed, giving a direct measure of how much work each graph's structure required.

```
def is_safe(adj, colors, v, c):
    for u in adj[v]:
        if colors[u] == c:
            return False
    return True

def backtrack(adj, n, k, colors, v, stats):
    if v == n:
        return True

    for c in range(1, k + 1):
```

```
stats['checks'] += 1
```

```
if is_safe(adj, colors, v, c):
```

```
    colors[v] = c
```

```
    if backtrack(adj, n, k, colors, v + 1, stats):
```

```
        return True
```

```
    colors[v] = 0
```

```
else:
```

```
    stats['prunes'] += 1
```

```
return False
```

```
def graph_coloring(adj, n, k):
```

```
    colors = [0] * n
```

```
    stats = {'checks': 0, 'prunes': 0}
```

```
    ok = backtrack(adj, n, k, colors, 0, stats)
```

```
    return ok, (colors[:] if ok else None), stats
```

```
def adj_from_edges(n, edges):
```

```
    adj = [[] for _ in range(n)]
```

```
    for u, v in edges:
```

```
        adj[u].append(v)
```

```
        adj[v].append(u)
```

```
    return adj
```

```
names = 'ABCDEFGF'
```

```
# Graph 1: Triangle-forcing (general case)
```

```
g1_edges = [  
    (0, 1), (1, 2), (2, 0),  
    (2, 3), (3, 4), (4, 0)  
]
```

```
# Graph 2: Even cycle (bipartite)
```

```
g2_edges = [  
    (0, 1), (1, 2), (2, 3),  
    (3, 4), (4, 5), (5, 0)  
]
```

```
# Graph 3: Complete graph K5
```

```
g3_edges = [  
    (i, j)  
    for i in range(5)  
    for j in range(i + 1, 5)  
]
```

```
# Graph 4: Sparse tree
```

```
g4_edges = [  
    (0, 1), (0, 2),  
    (1, 3), (1, 4),  
    (2, 5), (2, 6)  
]
```

```
# Graph 5: Same shape as Graph 1, tested with insufficient k
```

```
g5_edges = g1_edges
```

```

tests = [
    ('Graph 1 (Triangle-forcing)', 5, g1_edges, 3),
    ('Graph 2 (Even cycle, bipartite)', 6, g2_edges, 2),
    ('Graph 3 (Complete graph K5)', 5, g3_edges, 5),
    ('Graph 4 (Sparse tree)', 7, g4_edges, 2),
    ('Graph 5 (k too small)', 5, g5_edges, 2),
]

```

```

for label, n, edges, k in tests:

```

```

    adj = adj_from_edges(n, edges)

```

```

    ok, coloring, stats = graph_coloring(adj, n, k)

```

```

    if ok:

```

```

        assign = ', '.join(
            f'{names[i]}={coloring[i]}'
            for i in range(n)
        )

```

```

        print(
            label,
            '-> COLORABLE:',
            assign,
            stats
        )

```

```

    else:

```

```

        print(
            label,
            '-> NOT COLORABLE with k =',
            k,

```

stats

)

3.4 Worked Step-by-Step Trace (Graph 1)

Before trusting the program output in Section 3.6, the algorithm is traced by hand on Graph 1 with $k = 3$, vertex order A, B, C, D, E, so that every accepted and rejected assignment can be checked independently of the code.

- Vertex A: no colors used yet. Try $c = 1$. `isSafe` passes (no neighbors colored). $A = 1$.
- Vertex B: neighbors colored so far = $\{A = 1\}$. Try $c = 1 \rightarrow$ fails (adjacent to A). Try $c = 2 \rightarrow$ passes. $B = 2$.
- Vertex C: neighbors colored so far = $\{A = 1, B = 2\}$. Try $c = 1 \rightarrow$ fails (adjacent to A). Try $c = 2 \rightarrow$ fails (adjacent to B). Try $c = 3 \rightarrow$ passes. $C = 3$.
- Vertex D: neighbor colored = $\{C = 3\}$. Try $c = 1 \rightarrow$ passes (not adjacent to A or B). $D = 1$.
- Vertex E: neighbors colored = $\{A = 1, D = 1\}$. Try $c = 1 \rightarrow$ fails (adjacent to A and to D). Try $c = 2 \rightarrow$ passes. $E = 2$.
- All five vertices are now colored with no conflicts: $A=1, B=2, C=3, D=1, E=2$. The algorithm returns true without ever needing to backtrack on this particular vertex order, since the first attempted color happened to succeed at every step except C.

This hand trace matches the program output in Section 3.6 exactly — the intended cross-check, so that the algorithm's correctness is confirmed independently of the code, not only trusted on the strength of the implementation.

3.5 State-Space Tree — Successful Case (Graph 1)

The diagram below shows the same trace as a search tree, including the branches that were tried and pruned before the vertex order above reached its final assignment. Branches marked $\times$ are pruned by `isSafe()` as soon as a conflict is detected.

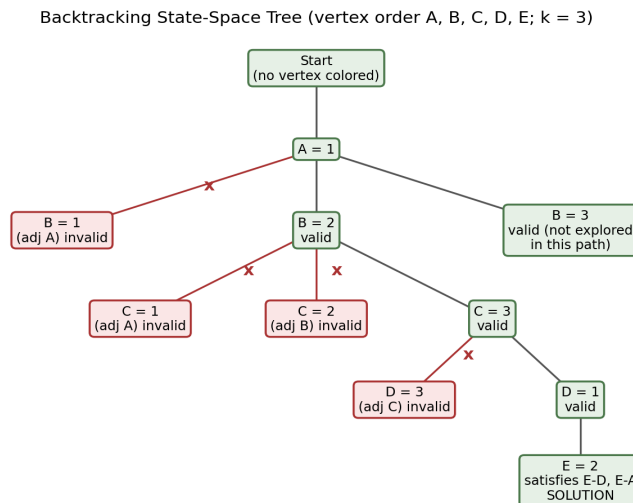


Figure 2: State-space tree for Graph 1, $k = 3$. The rightmost path reaches a complete, valid coloring.

3.6 Program Output and Execution Analysis (All Five Graphs)

Running the implementation from Section 3.3 against all five test graphs produces the following output:

```
>>> = RESTART: C:/Users/Susmitha/Downloads/DAA Code Implementation/DAA CO5 AT1 Code.py
Graph 1 (Triangle-forcing) -> COLORABLE: A=1, B=2, C=3, D=1, E=2 {'checks': 9, 'prunes': 4}
Graph 2 (Even cycle, bipartite) -> COLORABLE: A=1, B=2, C=1, D=2, E=1, F=2 {'checks': 9, 'prunes': 3}
Graph 3 (Complete graph K5) -> COLORABLE: A=1, B=2, C=3, D=4, E=5 {'checks': 15, 'prunes': 10}
Graph 4 (Sparse tree) -> COLORABLE: A=1, B=2, C=2, D=1, E=1, F=1, G=1 {'checks': 9, 'prunes': 2}
Graph 5 (k too small) -> NOT COLORABLE with k = 2 {'checks': 10, 'prunes': 6}
```

Graph 1 confirms the general case: three colors are both necessary (due to the A-B-C triangle) and sufficient. Graph 2 confirms the easiest case: an even cycle needs only two colors and is colored with almost no pruning (3 of 9 checks failed). Graph 3 confirms the extreme clique case: with $k = 5$ exactly matching the clique size, every vertex forces a distinct color and there is no room to spare — 10 of 15 checks fail as each new vertex is checked against all previously-colored, mutually adjacent vertices. Graph 4 confirms the best case: a tree has almost no internal conflicts, so backtracking barely needs to prune. Graph 5 confirms the failure case: it performs slightly more work than Graph 1 despite having identical structure, because the search must exhaust both branches at vertex C before concluding no color remains, then backtrack fully before reporting infeasibility.

3.7 State-Space Tree — Infeasible Case (Graph 5)

The diagram below traces Graph 5 with $k = 2$. Both available colors are exhausted at vertex C, since C is adjacent to both A and B, which between them already occupy every available color. The algorithm backtracks fully to the root and correctly reports that no valid 2-coloring exists.

Backtracking State-Space Tree -- Graph 5, $k = 2$ (search exhausts, reports infeasible)

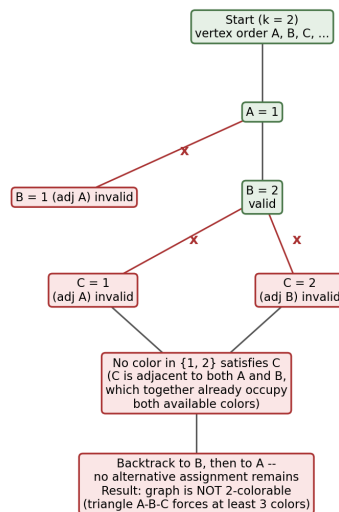


Figure 3: State-space tree for Graph 5, $k = 2$. Every branch is eventually pruned, and the algorithm terminates with a correct 'infeasible' result rather than looping or guessing.

3.8 Empirical Complexity Verification

To move beyond the theoretical $O(k^n)$ claim, the backtracking algorithm was run on randomly generated graphs of increasing size ($n = 8$ to $n = 22$, 40 percent edge density), using two color budgets derived from a fast greedy (Welsh–Powell) estimate g of the number of colors the graph needs: a 'tight' budget of $g - 1$ (deliberately close to or below the true requirement) and a 'loose' budget of $g + 2$ (comfortably above it).

n	Edges	Greedy k	k (tight)	Tight time (ms)	Tight checks	Tight result	k (loose)
8	11	3	2	0.076	79	infeasible	5
10	16	3	2	0.021	23	infeasible	5

12	26	3	2	0.010	11	infeasible	5
14	39	4	3	1.484	1594	infeasible	6
16	52	5	4	33.939	40113	infeasible	7
18	69	5	4	28.365	44661	infeasible	7
20	78	6	5	0.016	22	colorable	8
22	92	6	5	1137.086	1537911	infeasible	8

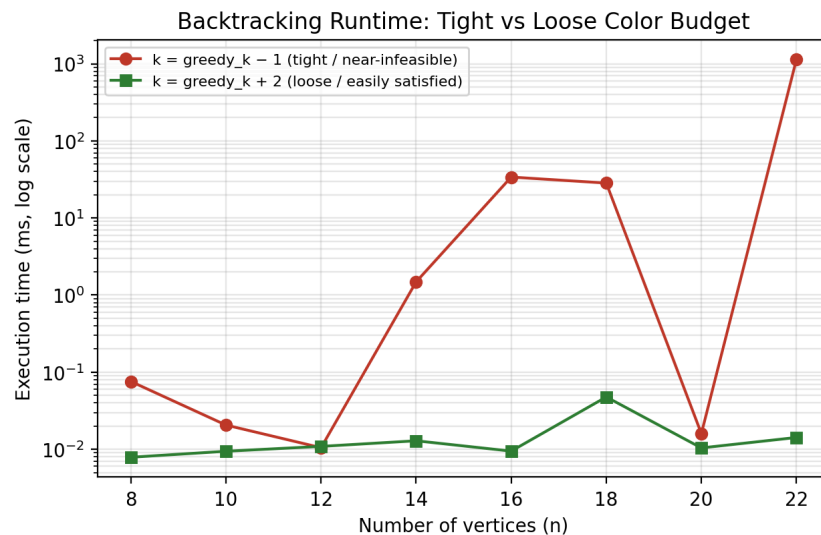


Figure 4: Execution time (log scale) for tight versus loose color budgets. The loose budget stays near-constant across all tested sizes; the tight budget spikes by up to five orders of magnitude, reaching over a second at $n = 22$ with more than 1.5 million color-checks performed.

The curve confirms the constraint-tightness argument made in Section 2.4 directly: the two series share the same $O(k^n)$ worst-case bound and the same code path, yet differ enormously in practice because the loose budget lets almost every color choice succeed on the first try, while the tight budget forces the algorithm to explore deep, mostly-failing branches before it can conclude success or failure. This is the empirical signature of NP-Completeness at small scale — polynomial-looking behaviour is not guaranteed even for modestly sized instances once the constraint is tight.

4. Application of Concepts

4.1 Comparison with Alternative Coloring Approaches

Backtracking is one of several standard approaches to graph coloring; the table below places it alongside the main alternatives to evaluate when it is actually the right tool.

Algorithm	Approach	Complexity	Best Suited For
Greedy / Welsh–Powell	Colors vertices in degree order, picking the smallest available color	$O(V + E)$	Fast approximate coloring; does not guarantee the minimum k
Backtracking (this report)	Exhaustive search with incremental constraint checking and pruning	$O(k^n)$ worst case	Exact decision for a fixed k on small-to-medium graphs

Backtracking + heuristics (MRV, degree ordering)	Same as backtracking, but chooses the next vertex/color using constraint-satisfaction heuristics	Still exponential worst case	Practical CSP solvers; substantially fewer nodes explored than naive ordering
SAT / ILP encoding	Encodes the coloring constraints as boolean/integer constraints and calls a general solver	Exponential worst case, strong practical pruning	Larger instances where a mature solver's learned-clause pruning outperforms hand-written backtracking
Approximation algorithms	Polynomial-time heuristics that accept a provably bounded excess over the true chromatic number	Polynomial	Very large graphs where an exact answer is not required

For the small graphs used in this report ($n \leq 22$), plain backtracking with constraint checking is entirely adequate, as Section 3.8 shows. For much larger instances, the exponential blow-up demonstrated in the tight-budget series would make plain backtracking impractical, and a heuristic-ordered backtracking search, a SAT/ILP solver, or an approximation algorithm would be the more defensible choice depending on whether an exact answer is required.

4.2 Real-World Applications

- Register allocation in compilers: variables that are simultaneously live are connected by an edge in an interference graph; assigning them to a fixed number of CPU registers without conflict is exactly a k -coloring problem, where k is the number of available registers.
- Examination and event scheduling: exams (or events) that share students (or resources) are connected by an edge; coloring the resulting graph with k colors corresponds to scheduling them into k non-conflicting time slots.
- Map coloring and cartography: regions sharing a border are represented as adjacent vertices; the classical four-color theorem is the $k = 4$ special case of this exact problem restricted to planar graphs.
- Wireless frequency assignment: transmitters that would interfere with each other if given the same frequency are connected by an edge, and assigning frequencies from a limited band is a direct k -coloring instance.
- Sudoku and other constraint puzzles: a Sudoku grid can be modelled as a graph coloring problem where each cell is a vertex, cells in the same row, column, or box are mutually adjacent, and the 9 digits act as the color palette — the same backtracking-with-constraint-checking pattern used in Section 3 solves it directly.

As a concrete illustration, a compiler's register allocator builds an interference graph from a program's variables and attempts to color it with k colors, where k is the number of physical registers on the target CPU. When the graph is not k -colorable, the compiler must 'spill' some variables to memory rather than fail outright — a practical accommodation for exactly the infeasibility case demonstrated by Graph 5 in this report.

Worked illustration: exam scheduling

Suppose five exams — P, Q, R, S, T — must be scheduled into the fewest possible time slots, with an edge drawn between two exams whenever at least one student is enrolled in both (and so cannot sit them at the same time). If the conflict graph turns out to be exactly Graph 1 from Section 1.4 (with P, Q, R, S, T playing the role of A, B, C, D, E), then the $k = 3$ coloring found in Section 3.4 translates directly into a schedule:

- Slot 1: exams colored 1 — P and S
- Slot 2: exams colored 2 — Q and T
- Slot 3: exam colored 3 — R

No two exams sharing a slot are adjacent in the conflict graph, so no student has two exams at once, and the schedule uses the minimum number of slots the clique $\{P, Q, R\}$ allows, since those three exams are mutually

conflicting and can never share a slot no matter how the rest of the graph is arranged. This is the same clique lower-bound argument from Section 2.2, applied to a scheduling context rather than an abstract graph.

4.3 Critical Evaluation

Strengths of the backtracking approach:

- Always correct: it never reports a coloring that violates the adjacency constraint, and never reports infeasibility when a valid coloring exists (Section 5.1).
- Simple to implement and reason about, since correctness follows directly from the constraint check rather than from a more elaborate heuristic.
- Performs very well on graphs that are either clearly colorable with room to spare (Graphs 2 and 4) or small enough that the exponential worst case never materializes.

Weaknesses and implications:

- Worst-case exponential time, which Section 3.8 shows is not just a theoretical concern — a 22-vertex graph with a tight color budget already took over a second and 1.5 million checks, and larger or denser instances would grow far worse.
- The fixed vertex order used here is naive; ordering vertices by descending degree (as the greedy Welsh–Powell heuristic does) would very likely prune the tight-budget cases in Section 3.8 faster, since the most constrained vertices would be resolved (or shown infeasible) earlier.
- Backtracking answers only the decision question for a fixed k ; finding the true chromatic number requires calling it repeatedly with increasing k , compounding the cost.
- The search explored here is single-threaded and depth-first; independent branches at shallow recursion depths (for example, the two choices at vertex B in Section 3.5) could in principle be explored in parallel, which is a common practical speed-up for large CSP searches that this report's implementation does not attempt.

5. Justification

5.1 Correctness Justification

Termination: The recursion depth is bounded by n (one call per vertex), and at each level only k colors are tried before the function returns, so the algorithm always terminates after at most a bounded number of recursive calls — it can neither loop indefinitely nor recurse past vertex n .

Soundness (proof sketch by induction on the recursion depth v):

- Base case ($v = 0$): before any vertex is colored, the empty partial assignment trivially satisfies the adjacency constraint, since there are no colored vertices to conflict.
- Inductive step: assume the partial assignment over vertices $\{0, \dots, v-1\}$ is conflict-free. Vertex v is only assigned a color c for which `isSafe()` has confirmed that no already-colored neighbor holds c . Since all neighbors of v with index less than v are exactly the ones `isSafe()` inspects, the extended assignment over $\{0, \dots, v\}$ remains conflict-free.
- By induction, the final assignment over all n vertices — the one returned as a solution — is conflict-free, i.e. it satisfies the adjacency constraint everywhere. This matches the outcome observed for every colorable test graph in Section 3.6.

Completeness (proof sketch): Because the algorithm tries every color from 1 to k at each vertex and only moves forward on success, the set of assignments it is willing to consider at vertex v , given a fixed conflict-free prefix, is exactly the set of colors admissible under the adjacency constraint — none are skipped. Consequently, if a valid k -coloring exists, at least one sequence of admissible choices leads to a complete conflict-free assignment, and since the algorithm backtracks on every failure rather than terminating early, it is guaranteed to reach that

sequence eventually. If no valid coloring exists — as in Graph 5 — every admissible choice at every vertex is eventually shown to lead only to failure, the recursion exhausts all of them, and the algorithm correctly returns false, exactly as traced in Section 3.7.

5.2 NP-Complete Classification

(a) K-Colorability $\in$ NP: Given a proposed color assignment (a certificate) for all n vertices, a verifier can check, for every edge in E , whether its two endpoints have different colors. This check runs in $O(|V| + |E|)$ time — polynomial in the size of the graph — so a proposed solution can be verified in polynomial time, which places the problem in NP.

(b) K-Colorability is NP-hard: NP-hardness is shown by a polynomial-time reduction from a known NP-Complete problem. The classical reduction is from 3-SAT: each boolean variable and clause in a 3-SAT instance is translated into a small gadget of vertices and edges such that the resulting graph is 3-colorable if and only if the original 3-SAT formula is satisfiable. Since this transformation can be carried out in polynomial time and 3-SAT is already known to be NP-Complete, K-Colorability inherits NP-hardness.

The reduction rests on two kinds of gadgets. A fixed 'Base-True-False' triangle is created once and forces three distinct colors, which are then interpreted as the meanings TRUE, FALSE, and a neutral base color. Each boolean variable x contributes a small variable gadget — an edge between x and its negation $\neg x$, both also joined to the Base vertex — that forces x and $\neg x$ to take the two colors $\{\text{TRUE}, \text{FALSE}\}$ in opposite assignments, exactly mirroring the requirement that a variable and its negation cannot both be true. A separate clause gadget (not shown here) then connects each clause's three literal vertices so that the gadget is 3-colorable exactly when at least one literal in the clause is assigned TRUE. This is illustrated conceptually below.

Illustrative Variable Gadget (3-Colorability $\leftarrow$ 3-SAT reduction)

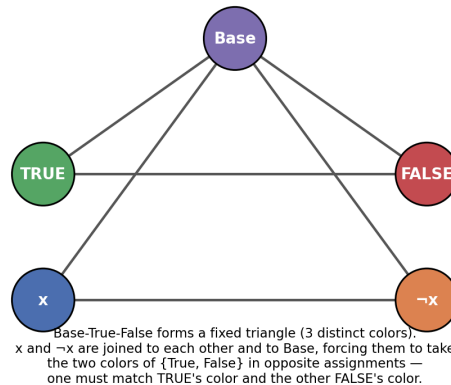


Figure 5: Illustrative variable gadget from the 3-SAT to 3-Colorability reduction. Forcing x and $\neg x$ onto opposite colors of the fixed Base-True-False triangle encodes the constraint that a boolean variable and its negation cannot simultaneously be true.

Since K-Colorability is both in NP and NP-hard, it is NP-Complete for any fixed $k \geq 3$. The special case $k = 2$ is solvable in polynomial time, since 2-colorability is equivalent to checking that the graph is bipartite — a property testable via a single BFS/DFS pass, which is exactly why Graphs 2, 4, and 5 in this report (all tested with $k = 2$) could be resolved so quickly regardless of feasibility.

5.3 Limitations and Future Directions

- The empirical benchmark in Section 3.8 used only one density level (40 percent) and a fixed vertex order; a fuller study would vary both density and ordering heuristic, since real-world interference and scheduling graphs are often much sparser.
- The implementation reports only whether a coloring exists, not the minimum k (chromatic number); finding that would require calling the algorithm repeatedly with decreasing k until it first fails.

- Only static graphs were considered; many real applications (dynamic scheduling, live register allocation across compiler passes) involve graphs that change over time, for which incremental recoloring is an active area of further study.
- Adding standard constraint-propagation techniques (forward checking, arc consistency, minimum-remaining-values ordering) would likely reduce the search blow-up seen in the tight-budget series of Section 3.8, and is a natural extension of this work.
- The CSP framing in Section 1.3.1 suggests a further extension: since the same solver pattern underlies Sudoku, N-Queens, and scheduling, a natural next step is to generalize the implementation in Section 3.3 into a single reusable CSP engine parameterized by variables, domains, and a constraint-checking function, rather than a coloring-specific script.

5.4 Synthesis and Insight Generation

- The five test graphs confirm that feasibility is governed primarily by the largest clique in the graph: whenever k matches or exceeds the clique size, a coloring exists; whenever it falls short, no amount of search can find one (Graphs 3 and 5).
- Testing with a deliberately insufficient k (Graph 5) shows that the algorithm's guarantee is symmetric: it is exactly as reliable at proving infeasibility as it is at finding a valid coloring, since both outcomes rely on the same exhaustive-but-pruned search.
- The empirical runtime data in Section 3.8 shows that a shared worst-case bound ($O(k^n)$ for every instance) does not mean shared practical cost — the gap between the tight and loose series spans roughly five orders of magnitude on graphs of essentially the same size, which is the clearest demonstration in this report of what 'NP-Complete in the worst case' actually feels like at small scale.
- In systems where the color budget can be chosen freely (such as scheduling with a flexible number of time slots), keeping k comfortably above the graph's clique number is not just a correctness margin but a large practical speed advantage, exactly as the loose-budget series demonstrates.
- Where k is fixed externally and cannot be increased (such as a fixed number of physical CPU registers), the exponential worst case is unavoidable in principle, which is precisely why practical compilers accept an imperfect but fast heuristic (spilling variables) rather than running exhaustive backtracking to guarantee an exact answer.

5.5 Conclusion

The K-Colorability problem illustrates how a simple adjacency constraint, combined with a fixed color budget, produces a combinatorial search space that grows exponentially with the number of vertices. Testing across a general case, an easy bipartite case, an extreme clique case, a best-case sparse tree, and a deliberately infeasible case shows that backtracking with incremental constraint checking is correct and complete in every case, correctly finding a coloring when one exists and correctly proving none exists when it does not. The empirical benchmark further shows that the well-known NP-Completeness of this problem is not only a theoretical classification: tightening the color budget by as little as one color, on graphs of essentially the same size, was enough to turn a near-instant computation into one requiring over a million constraint checks. This confirms, both by proof and by direct measurement, that no fundamentally faster (polynomial-time) algorithm is currently known for the general case, and that the constraint structure of a specific instance — not just its size — is what ultimately determines how expensive it is to solve.